

Highest Paid Toll

G R A F O S

Átila Silvio Carvalho Rocha Melo Oliveira 2320454

Lucas Barroso Sá 2426651

Victor Menezes do Vale 2325183

Modelagem e representação do grafo

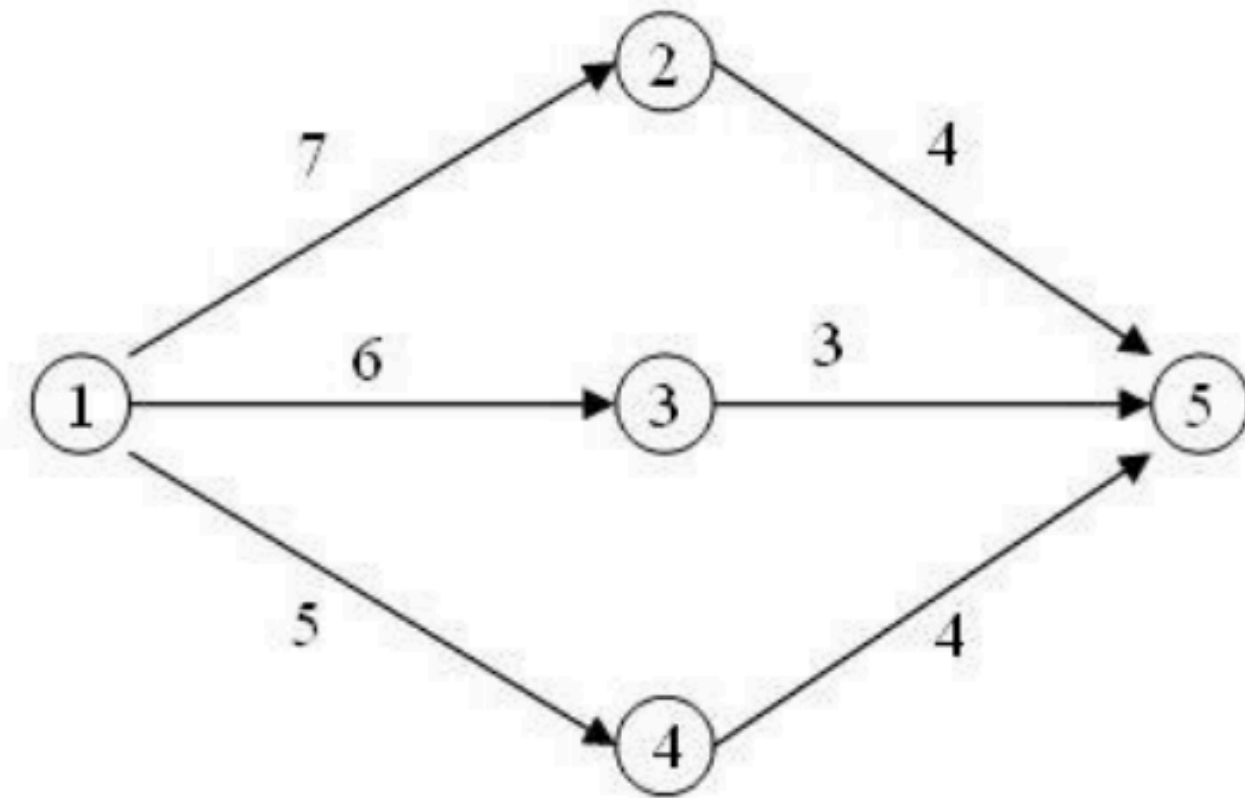
O Cenário: Ruas transformadas em vias de mão única com cobrança de pedágios para reduzir o trânsito.

A Missão: Viajar de um ponto de origem s até um destino t .

A Restrição: O custo total da viagem não pode estourar o orçamento financeiro p .

GRAFO

- Vértices: Locais da cidade.
- Arestas : Ruas de mão única.
- Pesos: Valor do pedágio.



DIJKSTRA

Para saber se uma rua específica pode fazer parte da viagem, aplicamos o algoritmo de Dijkstra duas vezes na fase de pré-processamento.

Etapa 1: O custo de ida :

Executamos o algoritmo de Dijkstra partindo da origem no grafo normal

Etapa 2: O custo de volta

Com um grafo reverso, invertendo a direção de todas as ruas. Rodamos o Dijkstra novamente, mas partindo do destino final.

```
for (int i = 0; i < M; i++) {  
    int u = StdIn.readInt();  
    int v = StdIn.readInt();  
    int c = StdIn.readInt();  
  
    DirectedEdge e = new DirectedEdge(u, v, c);  
  
    grafo.addEdge(e);  
    reverso.addEdge(new DirectedEdge(v, u, c));  
  
    arestas.add(e);  
}
```

```
DijkstraSP caminhoSaindoDeS = new DijkstraSP(grafo, s);  
DijkstraSP caminhoAteT = new DijkstraSP(reverso, t);
```

DIJKSTRA

Etapa 3: A verificação do orçamento
testamos cada rua individualmente. Uma rua só
é válida se a soma for menor ou igual ao
orçamento total:

O custo para chegar até ela +

O valor do pedágio dela +

O custo para ir dela até o destino.

```
int resposta = -1;

for (DirectedEdge e : arestas) {
    int u = e.from();
    int v = e.to();
    int c = (int) e.weight();

    if (caminhoSaindoDeS.hasPathTo(u) && caminhoAteT.hasPathTo(v)) {
        double custoTotal = caminhoSaindoDeS.distTo(u) + c + caminhoAteT.distTo(v);

        if (custoTotal <= p) {
            resposta = Math.max(resposta, c);
        }
    }
}

StdOut.println(resposta);
```

Etapa 4: A escolha do maior pedágio

Enquanto testa todas as ruas disponíveis, o algoritmo
guarda e retorna apenas o maior valor de pedágio
encontrado entre todas as rotas que respeitaram o dinheiro
disponível.

ANÁLISE DE COMPLEXIDADE

COMPLEXIDADE DE TEMPO: $O(E \log V)$

DIJKSTRA DUPLO: $2 * O(E \log V)$
(GARANTIDO PELO USO DE FILA DE PRIORIDADE
COM MIN-HEAP).

VALIDAÇÃO FINAL: $O(E)$ (UM ÚNICO LAÇO
ITERANDO SOBRE AS ARESTAS).

**TEMPO TOTAL: O TERMO LOGARÍTMICO
DOMINA, RESULTANDO EM $O(E \log V)$.**

ANÁLISE DE COMPLEXIDADE

COMPLEXIDADE DE ESPAÇO: $O(V + E)$

ARMAZENAMENTO: GRAFOS (ORIGINAL E REVERSO) REPRESENTADOS POR LISTAS DE ADJACÊNCIA $O(E)$

ESTRUTURAS AUXILIARES: ARRAYS DE DISTÂNCIAS E FILA DE PRIORIDADE OCUPAM $O(V)$.

MEMÓRIA TOTAL: $O(E + V)$ LINEAR E PROPORCIONAL AO TAMANHO DA CIDADE.

ENVIO NO ONLINE JUDGE



Online Judge
Home > My Submissions

ENHANCED BY Google

Search

Main Menu

- Home
- My Account
- Contact Us
- ICPC Live Archive
- Logout

Online Judge

- Quick Submit
- Migrate submissions
- My Submissions**
- My Statistics
- My uHunt with Virtual Contest Service
- Browse Problems
- Quick access, info and search

My Submissions

#	Problem	Verdict	Language	Run Time	Submission Date
31158942	12047 Highest Paid Toll	Accepted	JAVA	0.870	2026-06-01 20:26:53
31158939	12047 Highest Paid Toll	Compilation error	JAVA	0.000	2026-06-01 20:21:57
31158938	12047 Highest Paid Toll	Compilation error	JAVA	0.000	2026-06-01 20:19:51

<< Start < Prev 1 Next > End >>

Display # 30 ▾ Results 1 - 3 of 3

Our Patreons

Diamond Sponsors

Steven & Felix Halim

Reinardus Pradhitya

Gold Sponsors

--- YOUR NAME HERE ----

Silver Sponsors

--- YOUR NAME HERE ----

Bronze Sponsors

Christianto Handojo

Krzysztof Adamek

O B R I G A D O !